TASK 15 — Server Monitoring Using Nagios

Objective

To install and configure Nagios for server monitoring and to study the commands used to monitor the server and its services.

PART 1: INSTALL NAGIOS SERVER

Step 1 — Update the System

Update the system packages before installing Nagios.

sudo dnf update -y

Step 2 — Install Required Software

Install the packages required to compile and run Nagios.

sudo dnf install -y autoconf gcc glibc make wget unzip httpd php php-cli php-gd gd gd-devel openssl openssl-devel

These packages are required for compiling Nagios, running the web interface, downloading files, generating graphs, and handling secure connections.

Step 3 — Start the Web Server

Start Apache/httpd and enable it to start automatically after reboot.

sudo systemctl enable httpd --now

Apache/httpd provides the web server required to access the Nagios web interface.

Step 4 — Create a Dedicated Nagios User and Group

Create the Nagios user and the shared nagcmd group.

sudo useradd nagios

sudo groupadd nagcmd

sudo usermod -a -G nagcmd nagios

sudo usermod -a -G nagcmd apache

The nagios user is used to run Nagios, while the nagcmd group allows Apache to communicate with Nagios through the command interface.

Step 5 — Download and Compile Nagios Core

Download Nagios Core source code:

cd /tmp

wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.7.tar.gz

Extract the downloaded archive:

tar -xvzf nagios-4.5.7.tar.gz

Enter the Nagios directory:

cd nagios-4.5.7

Configure Nagios:

./configure --with-nagios-group=nagios --with-command-group=nagcmd

Compile Nagios:

make all

Install the Nagios components:

sudo make install

sudo make install-init

sudo make install-config

sudo make install-commandmode

sudo make install-webconf

These installation commands install the Nagios monitoring engine, startup configuration, default configuration files, command interface, and Apache web configuration.

Step 6 — Set a Web Login Password

Create the Nagios web administrator password:

sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin

The username used for the Nagios web interface is:

nagiosadmin

A password is entered when prompted.

Step 7 — Install Nagios Plugins

Nagios plugins perform the actual monitoring checks.

Download the plugins:

cd /tmp

wget https://nagios-plugins.org/download/nagios-plugins-2.4.6.tar.gz

Extract the archive:

tar -xvzf nagios-plugins-2.4.6.tar.gz

Enter the directory:

cd nagios-plugins-2.4.6

Configure:

./configure --with-nagios-user=nagios --with-nagios-group=nagios

Compile and install:

make

sudo make install

Nagios itself is the monitoring engine, while plugins perform checks such as CPU load, disk space and ping.

Step 8 — Start Nagios and Restart the Web Server

Start and enable Nagios:

sudo systemctl enable nagios --now

Restart Apache/httpd:

sudo systemctl restart httpd

This starts Nagios and allows Apache to load the Nagios web configuration.

Step 9 — Open the Firewall for Web Access

Allow HTTP traffic through the firewall:

sudo firewall-cmd --permanent --add-service=http

sudo firewall-cmd --reload

This allows access to the Nagios web interface through HTTP port 80.

Step 10 — Log in to Nagios

Open a browser and access:

http://<your-server-ip>/nagios

Login username:

nagiosadmin

Use the password created in Step 6.

PART 2: SET UP THE CLIENT

The client is the machine that Nagios will monitor.

Step 11 — Install EPEL

Install the Extra Packages for Enterprise Linux repository:

sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm

EPEL provides additional packages such as NRPE that are not included in the default RHEL package sources.

Step 12 — Install NRPE and Plugins

Install NRPE and the required monitoring plugins:

sudo dnf install -y nrpe nagios-plugins-load nagios-plugins-disk nagios-plugins-users nagios-plugins-procs

NRPE allows the Nagios server to request monitoring information from the client. The plugins perform the actual checks on the client.

Step 13 — Allow the Nagios Server

Edit the NRPE configuration:

sudo nano /etc/nagios/nrpe.cfg

Configure:

allowed_hosts=127.0.0.1,::1,<NAGIOS-SERVER-IP>

This allows the Nagios server's IP address to query the NRPE agent.

Step 14 — Define NRPE Checks

In /etc/nagios/nrpe.cfg, configure the load check:

command[check_load]=/usr/lib64/nagios/plugins/check_load -w 2.0,1.5,1.0 -c 3.0,2.5,2.0

Configure the disk check:

command[check_disk]=/usr/lib64/nagios/plugins/check_disk -w 20% -c 10% -p /

The -w values represent warning thresholds and the -c values represent critical thresholds.

The CPU core count can be checked with:

nproc

For a 2-core machine, the configured load thresholds correspond to the CPU capacity described in the task.

Step 15 — Start NRPE

Enable and start the NRPE service:

sudo systemctl enable nrpe --now

NRPE starts listening for monitoring requests from the Nagios server.

Step 16 — Open NRPE Firewall Port

NRPE communicates through port 5666.

Allow the port:

sudo firewall-cmd --permanent --add-port=5666/tcp

sudo firewall-cmd --reload

Step 17 — Open Port 5666 in AWS Security Group

In the AWS Console:

EC2

→ Instance

→ Security

→ Security Group

→ Inbound Rules

→ Add Rule

Configure:

Type: Custom TCP

Port: 5666

Source: <NAGIOS-SERVER-IP>

Both the AWS Security Group and the client's operating-system firewall must allow the NRPE connection

PART 3: CONNECT THE NAGIOS SERVER TO THE CLIENT

Step 18 — Install the NRPE Plugin on the Nagios Server

Install the plugin that allows the Nagios server to communicate with NRPE:

sudo dnf install -y nagios-plugins-nrpe

This installs check_nrpe, which Nagios uses to communicate with the client's NRPE agent.

Step 19 — Configure the Plugin Path

Create the required symbolic link:

sudo ln -s /usr/lib64/nagios/plugins/check_nrpe /usr/local/nagios/libexec/check_nrpe

This allows Nagios to find the check_nrpe plugin in its expected plugin directory.

Step 20 — Test the NRPE Connection

Test communication between the Nagios server and client:

/usr/local/nagios/libexec/check_nrpe -H <CLIENT-IP>

A working response confirms that the Nagios server can communicate with the client's NRPE agent.

Step 21 — Define the NRPE Command in Nagios

Edit:

sudo nano /usr/local/nagios/etc/objects/commands.cfg

Add:

```
define command{
 command_name check_nrpe
 command_line $USER1$/check_nrpe -H $HOSTADDRESS$ -c $ARG1$
}
```

This defines how Nagios executes the check_nrpe command.

Step 22 — Define the Client and Services

Create the client configuration:

sudo nano /usr/local/nagios/etc/objects/client1.cfg

Add:

```
define host{
 use linux-server
 host_name client1
 alias Client Server
 address <CLIENT-IP>
 max_check_attempts 5
 check_period 24x7
 notification_interval 30
 notification_period 24x7
}

define service{
 use generic-service
 host_name client1
 service_description Current Load
 check_command check_nrpe!check_load
}

define service{
 use generic-service
```

host_name client1

service_description Root Partition

check_command check_nrpe!check_disk

}

This tells Nagios to monitor the client and check its Current Load and Root Partition using NRPE.

Step 23 — Register the Client Configuration

Edit the main Nagios configuration:

sudo nano /usr/local/nagios/etc/nagios.cfg

Add:

cfg_file=/usr/local/nagios/etc/objects/client1.cfg

This tells Nagios to load the new client configuration file.

Step 24 — Check the Nagios Configuration

Before restarting Nagios, check the configuration:

sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg

This performs a configuration check. The expected result is:

Total Errors: 0

Step 25 — Restart Nagios

Apply the new configuration:

sudo systemctl restart nagios

Step 26 — Check the Nagios Web Interface

Open:

http://<server-ip>/nagios

Go to Hosts.

The Nagios web interface should show:

localhost

client1

The configured services for client1 include:

Current Load

Root Partition

PART 4: LOAD TESTING

Step 27 — Check Current CPU Load

Check the current load average:

uptime

The command displays the load average for the last 1, 5 and 15 minutes.

Step 28 — Check CPU Core Count

Check the number of CPU cores:

nproc

This helps understand the relationship between CPU capacity and load average.

Step 29 — Install Stress Testing Tool

Install stress-ng:

sudo dnf install -y stress-ng

Step 30 — Generate CPU Load

Generate test CPU load:

stress-ng --cpu 2 --timeout 60s

This creates temporary CPU load so that the monitoring system can observe the change.

Step 31 — Monitor the Load

Monitor the load continuously:

watch -n 1 uptime

This refreshes the load information every second so that the increase can be observed.

Step 32 — Stop the Load Test

Stop the stress-ng process:

pkill stress-ng

This stops the artificial CPU load and allows the system to return to its normal state.

Step 33 — Confirm the Load Test Has Stopped

Check whether stress-ng is still running:

pgrep stress-ng

No output indicates that the process is no longer running.

Step 34 — Force a Nagios Check

From the Nagios web interface:

Service

→ Re-schedule the next check of this service

→ Force Check

→ Commit

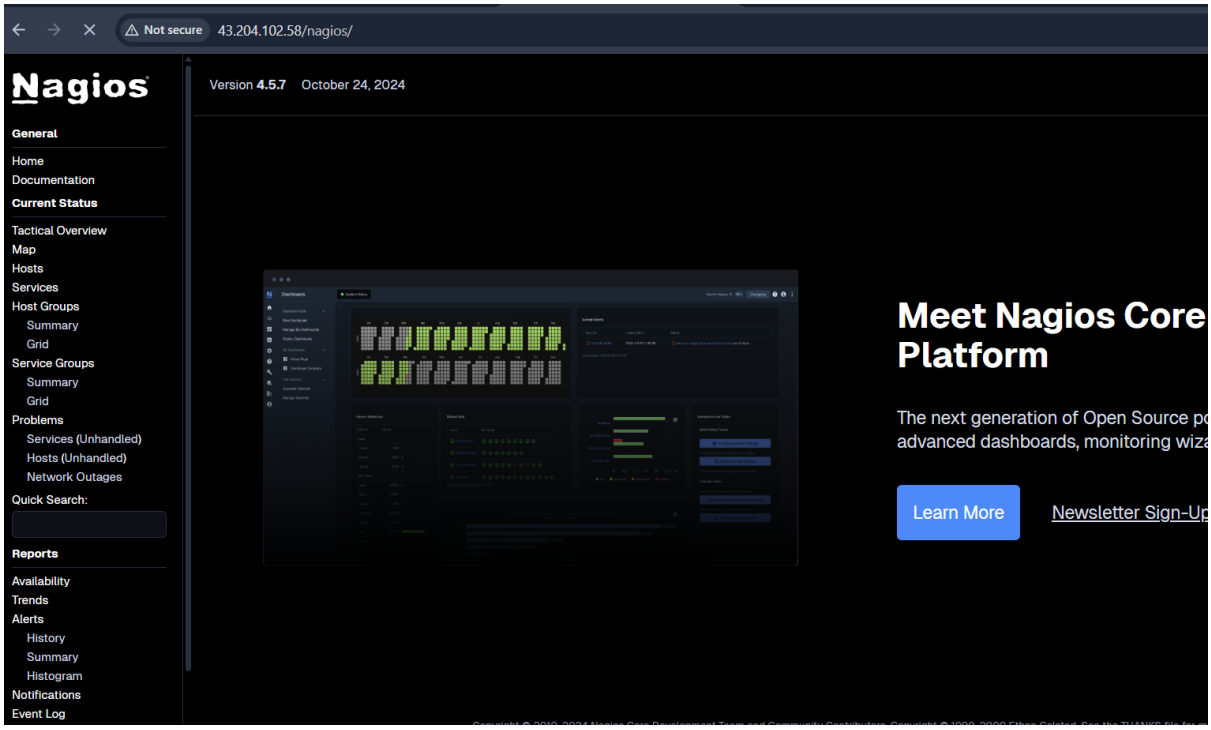
This forces Nagios to check the service immediately instead of waiting for the normal monitoring interval.

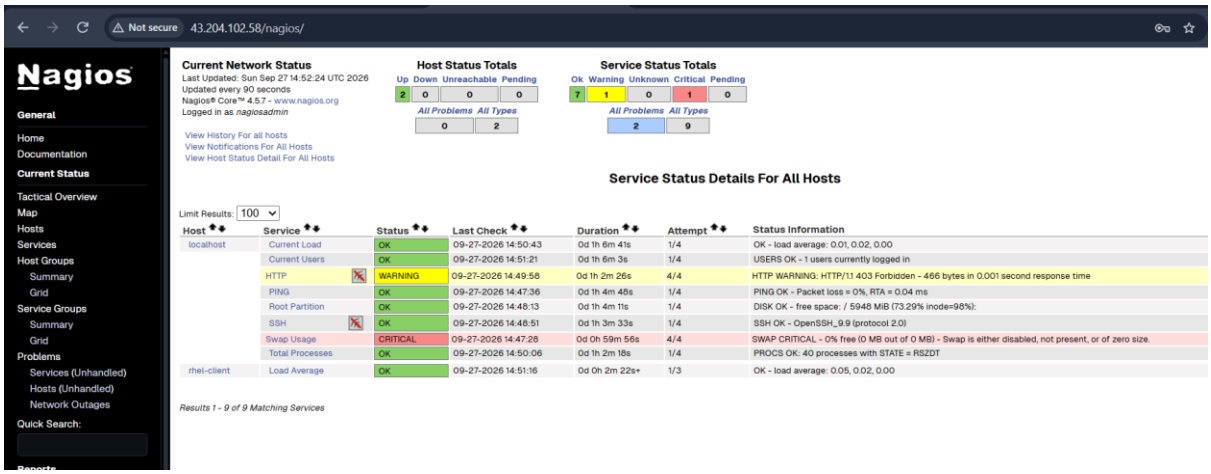
Result

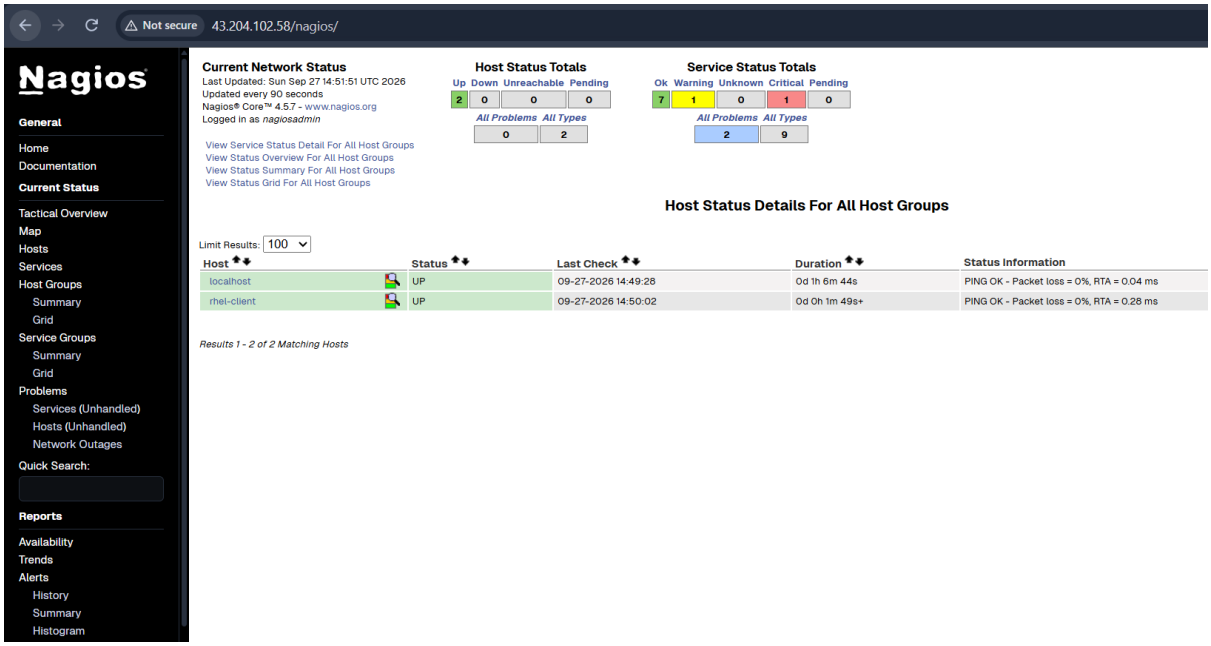
Nagios was installed and configured as a server monitoring tool. The Nagios server was connected to a client using NRPE, and monitoring checks were configured for:

- CPU Load
- Root Partition

The Nagios web interface was configured to display the monitored client, and CPU load was artificially generated to observe the monitoring system reacting to a change. The task also used AWS Security Group and firewall rules to allow NRPE communication on port 5666.






Not secure
43.204.102.58/nagios/
Nagios
General
Home
Documentation
Current Status
Tactical Overview
Map
Hosts
Services
Host Groups
Summary
Grid
Service Groups
Summary
Grid
Problems
Services (Unhandled)
Hosts (Unhandled)
Network Outages
Quick Search:
Reports
Availability
Trends
Alerts
History
Summary
Histogram
Current Network Status
Last Updated: Sun Sep 27 14:51:51 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.5.7 - www.nagios.org
Logged in as nagiosadmin
View Service Status Detail For All Host Groups
View Status Overview For All Host Groups
View Status Summary For All Host Groups
View Status Grid For All Host Groups
Host Status Totals
Up Down Unreachable Pending
2 0 0 0
All Problems All Types
0 2
Service Status Totals
Ok Warning Unknown Critical Pending
7 1 0 1 0
All Problems All Types
2 9
Host Status Details For All Host Groups
Limit Results: 100
Host
Status
Last Check
Duration
Status Information
localhost
UP
09-27-2026 14:49:28
0d 1h 6m 44s
PING OK - Packet loss = 0%, RTA = 0.04 ms
rhel-client
UP
09-27-2026 14:50:02
0d 0h 1m 49s+
PING OK - Packet loss = 0%, RTA = 0.28 ms
Results 1 - 2 of 2 Matching Hosts

Zabbix Server Monitoring – Installation and Configuration

Objective

- Install and check the functioning of the server monitoring tool Zabbix
- Install the Zabbix agent on a client machine and monitor it from the Zabbix server
- Test the monitoring by generating load and by stopping a monitored service
- Study in detail the commands used to monitor the server as well as the services

Requirements

- Two AWS EC2 instances running RHEL 9 (one Zabbix server, one client)
- Zabbix 7.0 server, web frontend and agent
- MariaDB database server
- Apache (httpd) and PHP-FPM for the web frontend
- AWS Security Group configuration (ports 22, 80, 10050 and 10051)

Procedure

Step 1 – Launch the EC2 Instances

Two EC2 instances running RHEL 9.8 were launched. One was named server (private IP 172.31.6.204) and hosted Zabbix. The other was named client (private IP 172.31.5.66) and was the machine to be monitored. Both were placed in the same VPC so that they could communicate using private IP addresses.

Step 2 – Add Swap Space on the Server

During the first package installation, the process was terminated with a "Killed" message because the instance had only about 1 GB of RAM. A 3 GB swap file was created and made permanent to solve this.

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=3072 status=progress
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

The swap space was confirmed using free -h.

Step 3 – Add the Zabbix Repository and Install the Packages

The official Zabbix 7.0 repository for RHEL 9 was added, and the Zabbix server, web frontend, Apache configuration, SELinux policy, agent and MariaDB were installed.

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-7.0-2.el9.noarch.rpm
sudo dnf clean all
sudo dnf install -y zabbix-server-mysql zabbix-web-mysql zabbix-apache-conf \
  zabbix-sql-scripts zabbix-selinux-policy zabbix-agent mariadb-server --disablerepo=epel
```

The installation was verified with rpm -q zabbix-server-mysql mariadb-server.

Step 4 – Start and Secure the Database

```
sudo systemctl enable mariadb --now
sudo mysql_secure_installation
```

A root password was set, and anonymous users, remote root login and the test database were removed.

Step 5 – Create the Zabbix Database and User

```
sudo mysql -uroot -p
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER zabbix@localhost IDENTIFIED BY '<password>';
GRANT ALL PRIVILEGES ON zabbix.* TO zabbix@localhost;
SET GLOBAL log_bin_trust_function_creators = 1;
QUIT;
```

Step 6 – Import the Zabbix Schema

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
```

The first attempt failed with an "Access denied" error, so the password of the zabbix database user was reset. A later attempt showed "Table already exists", which meant an earlier import had run only partly. The incomplete database was therefore dropped and recreated, and the import was run once again successfully. The import was verified, and the temporary setting was switched off:

```
mysql -uzabbix -p zabbix -e "SELECT COUNT(*) FROM users;"
sudo mysql -uroot -p -e "SET GLOBAL log_bin_trust_function_creators = 0;"
```

The query returned 2, which are the default Admin and guest users.

Step 7 – Configure the Zabbix Server and Start the Services

The database password was set in the Zabbix server configuration file /etc/zabbix/zabbix_server.conf:

DBPassword=<password>

The services were started and enabled:

```
sudo systemctl enable --now zabbix-server zabbix-agent httpd php-fpm
systemctl is-active zabbix-server zabbix-agent httpd php-fpm
```

All four services reported **active**. The OS firewall (firewalld) was inactive, so access was controlled by the AWS Security Group.

Step 8 – Complete the Web Setup

The Zabbix web installation wizard was opened in the browser:

http://<Server-Public-IP>/zabbix

The pre-requisite check passed, and the database connection was configured with type MySQL, host localhost, database name zabbix and user zabbix. The time zone was set to Asia/Kolkata. After the installation finished, login was done with the default Admin account.

Step 9 – Set Up the Client Machine

Swap space (2 GB) was added on the client in the same way as on the server. The Zabbix repository was added, and the agent was installed:

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-7.0-2.el9.noarch.rpm
sudo dnf clean all
sudo dnf install -y zabbix-agent --disablerepo=epel
```

The agent configuration file /etc/zabbix/zabbix_agentd.conf was edited:

```
Server=172.31.6.204
ServerActive=172.31.6.204
Hostname=client1
```

The agent was started and its listening port was verified:

```
sudo systemctl enable --now zabbix-agent
systemctl is-active zabbix-agent
sudo ss -tlnp | grep 10050
```

The agent was active and listening on port 10050.

Step 10 – Configure the AWS Security Group

The first connection test from the server timed out, which showed that the traffic was blocked at the network level. Inbound rules for TCP port 10050 (server to client) and TCP port 10051 (client to server) were added to the Security Group. Port 22 (SSH) and port 80 (web interface) were also allowed.

Step 11 – Test the Connection from the Server

sudo dnf install -y zabbix-get
zabbix_get -s 172.31.5.66 -k agent.ping

The command returned **1**, confirming that the server could communicate with the client's agent.

Step 12 – Add the Client as a Host in Zabbix

In the web interface, **Data collection → Hosts → Create host** was used with these settings:

- Host name: client1
- Template: Linux by Zabbix agent
- Host group: Linux servers
- Interface: Agent, IP address 172.31.5.66, port 10050

After one to two minutes, the **ZBX availability icon turned green**.

Step 13 – Load Test

The baseline load was checked under **Monitoring → Latest data** (host client1, name "load"). Two CPU-intensive processes were then started on the client:

yes > /dev/null &
yes > /dev/null &
uptime

The load average on the client rose from 0.00 to about 1.75 (uptime), and Zabbix showed the Load average (1m avg) rising to 1.95. The processes were then stopped:

pkill yes
pgrep yes

The load average then fell back toward normal, which was visible in the Zabbix graph.

Step 14 – Alert Test

To test failure detection, the agent was stopped on the client:

```
sudo systemctl stop zabbix-agent
systemctl is-active zabbix-agent
```

After a few minutes the ZBX icon turned **red**, and a problem was shown under **Monitoring → Problems**. The agent was then started again:

```
sudo systemctl start zabbix-agent
```

The icon turned green again and the problem was resolved.

Monitoring Commands Studied

Zabbix item keys tested from the server using zabbix_get:

```
zabbix_get -s 172.31.5.66 -k agent.ping
zabbix_get -s 172.31.5.66 -k system.cpu.load[all,avg1]
zabbix_get -s 172.31.5.66 -k vm.memory.size[available]
zabbix_get -s 172.31.5.66 -k vfs.fs.size[/,pfree]
zabbix_get -s 172.31.5.66 -k system.uptime
```

Server resource monitoring:

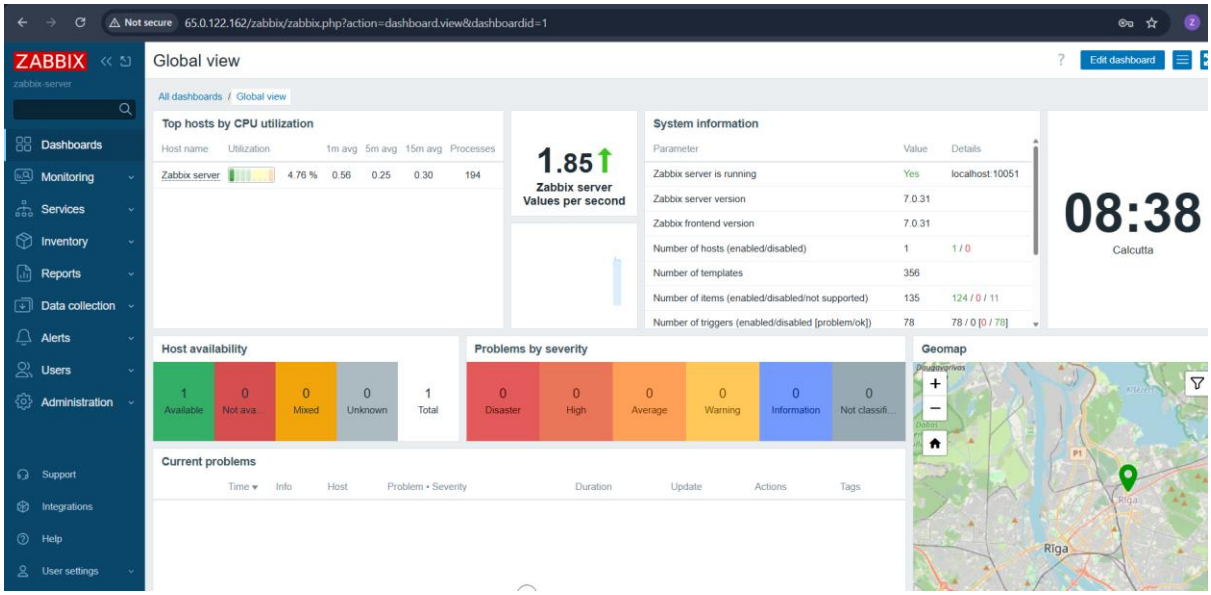
```
uptime                  # load average (1, 5, 15 minutes)
free -h                 # memory and swap usage
df -h                   # disk space
top -bn1 | head -15     # running processes and CPU use
```

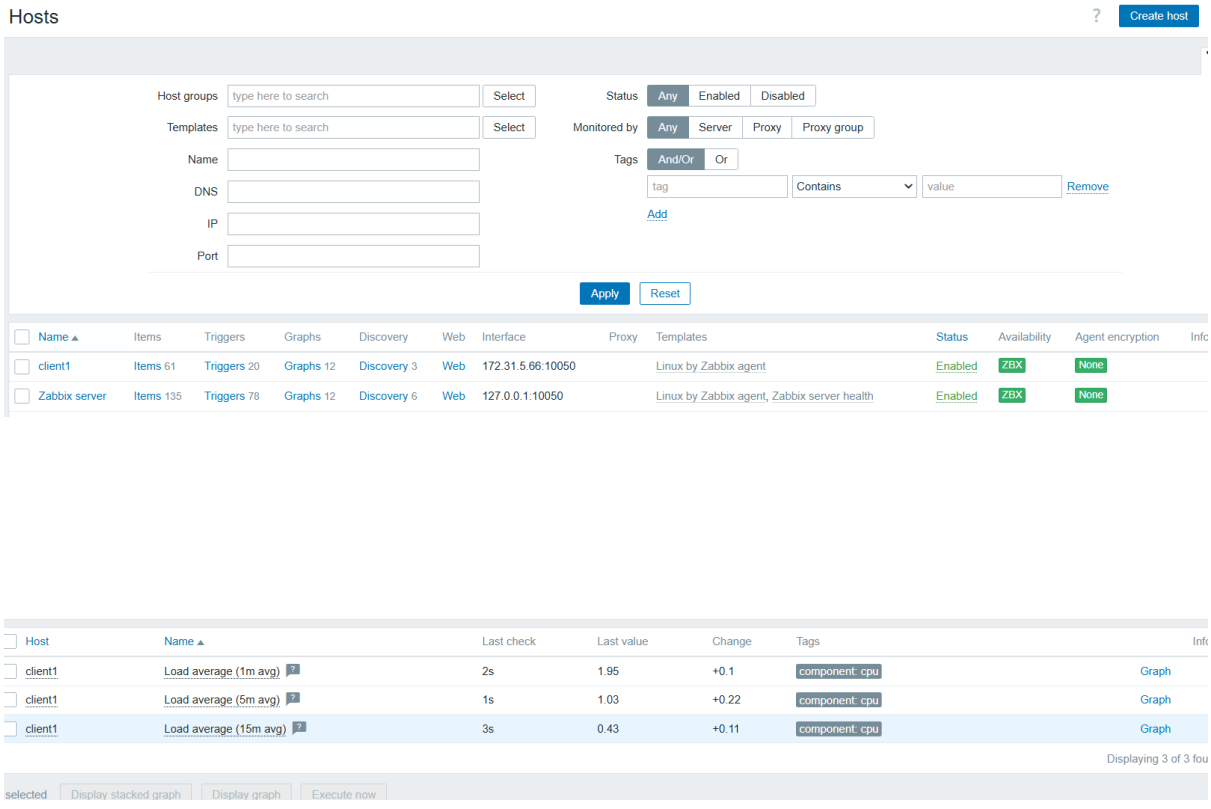
Service monitoring:

```
systemctl status zabbix-agent
systemctl is-active httpd
ss -tlnp                # listening ports
```

Network and troubleshooting:

```
ping -c 4 172.31.5.66
timeout 5 bash -c "</dev/tcp/172.31.5.66/10050" && echo "Port is OPEN" || echo "Port is CLOSED"
sudo tail -20 /var/log/zabbix/zabbix_server.log
sudo grep -E "^(Server|ServerActive|Hostname)=" /etc/zabbix/zabbix_agentd.conf
```


Not secure 65.0.122.162/zabbix/zabbix.php?action=dashboard.view&dashboardid=1
ZABBIX
zabbix-server
Global view
Edit dashboard
All dashboards / Global view
Dashboards
Monitoring
Services
Inventory
Reports
Data collection
Alerts
Users
Administration
Support
Integrations
Help
User settings
Top hosts by CPU utilization
Host name Utilization 1m avg 5m avg 15m avg Processes
Zabbix server 4.76 % 0.56 0.25 0.30 194
1.85
Zabbix server
Values per second
System information
Parameter Value Details
Zabbix server is running Yes localhost:10051
Zabbix server version 7.0.31
Zabbix frontend version 7.0.31
Number of hosts (enabled/disabled) 1 1 / 0
Number of templates 356
Number of items (enabled/disabled/not supported) 135 124 / 0 / 11
Number of triggers (enabled/disabled [problem/ok]) 78 78 / 0 [0 / 78]
08:38
Calcutta
Host availability
1 Available
0 Not ava...
0 Mixed
0 Unknown
1 Total
Problems by severity
0 Disaster
0 High
0 Average
0 Warning
0 Information
0 Not classifi...
Geomap
Riga
Current problems
Time Info Host Problem • Severity Duration Update Actions Tags


Hosts
Create host
Host groups type here to search Select
Templates type here to search Select
Name
DNS
IP
Port
Status Any Enabled Disabled
Monitored by Any Server Proxy Proxy group
Tags And/Or Or
tag Contains value Remove
Add
Apply Reset
Name Items Triggers Graphs Discovery Web Interface Proxy Templates Status Availability Agent encryption Info
client1 Items 61 Triggers 20 Graphs 12 Discovery 3 Web 172.31.5.66:10050 Linux by Zabbix agent Enabled ZBX None
Zabbix server Items 135 Triggers 78 Graphs 12 Discovery 6 Web 127.0.0.1:10050 Linux by Zabbix agent, Zabbix server health Enabled ZBX None

Host Name Last check Last value Change Tags
client1 Load average (1m avg) 2s 1.95 +0.1 component: cpu Graph
client1 Load average (5m avg) 1s 1.03 +0.22 component: cpu Graph
client1 Load average (15m avg) 3s 0.43 +0.11 component: cpu Graph
Displaying 3 of 3 fou
selected Display stacked graph Display graph Execute now

Verification

- Two RHEL 9 EC2 instances were launched for the Zabbix server and the client.
- Swap space was added to prevent installations from being killed due to low memory.
- The Zabbix server, web frontend, agent and MariaDB were installed successfully.
- The Zabbix database was created and the schema imported successfully.
- All Zabbix services (zabbix-server, zabbix-agent, httpd, php-fpm) were active.
- The Zabbix web interface was accessible and the login was successful.
- The Zabbix agent was installed and configured on the client and listened on port 10050.
- The AWS Security Group was configured to allow ports 22, 80, 10050 and 10051.
- The server communicated with the client agent (zabbix_get returned 1).
- The client was added as a host and its availability showed green.
- The load test was monitored through the Latest data page and graphs.
- Stopping the agent produced a red status and a problem, and restoring it cleared the problem.
- The commands used to monitor the server and its services were studied and tested.

Conclusion

Through this task, the Zabbix 7.0 monitoring server was successfully installed and configured on an AWS EC2 instance running RHEL 9, with MariaDB as the database and Apache as the web server. A second EC2 instance was set up as a client with the Zabbix agent and was added to the server as a monitored host. The monitoring was tested by generating CPU load, which was observed in the Zabbix graphs, and by stopping the agent, which produced a visible alert that cleared after recovery. Problems found during the work, such as low memory causing the installation to be killed, a partial database import and a blocked agent port, were identified and fixed. This task provided practical, hands-on experience with server monitoring, agent-based host monitoring, AWS Security Group configuration and the commands used to monitor servers and services.